RELAZIONE PROGETTO E-COMMERCE

Sommario

1. INTRODUZIONE....2
2. FUNZIONALITÀ PRINCIPALI....3
3. PROGETTAZIONE CONCETTUALE E LOGICA....5
Progettazione concettuale:....5
Progettazione logica:....7
4. QUERY PRINCIPALI....8
SQLAlchemy:....8
5. PRINCIPALI SCELTE PROGETTUALI....9
CHECK:....9
TRIGGER:....10
INDICI:....10
6. ULTERIORI INFORMAZIONI....10
7. CONTRIBUTO DEL PROGETTO....11

1. INTRODUZIONE

L’applicazione in questione è una piattaforma di E-commerce che tratta prodotti utili a chi lavora nell’ambito edilizio e falegnameria.

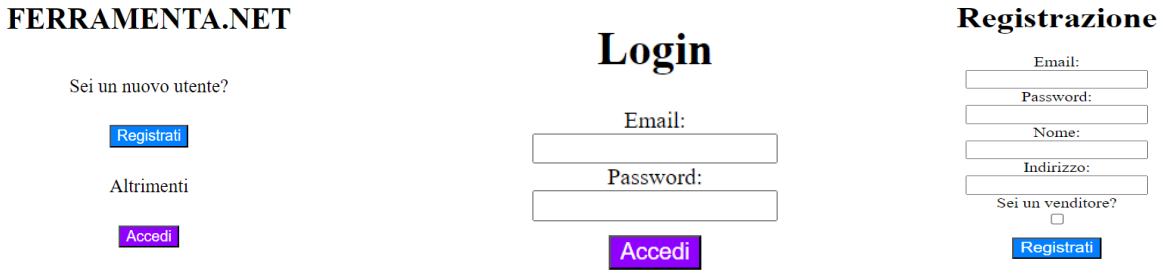
È possibile selezionare i prodotti e le relative quantità da aggiungere al carrello e procedere con il pagamento, questo per quanto riguarda l’acquirente.

Per il venditore ci sono le possibilità di aggiungere nuovi prodotti con descrizione, disponibilità e immagine. Inoltre si può modificare la disponibilità dei prodotti già presenti ed eventualmente rimuoverli.

2. FUNZIONALITÀ PRINCIPALI

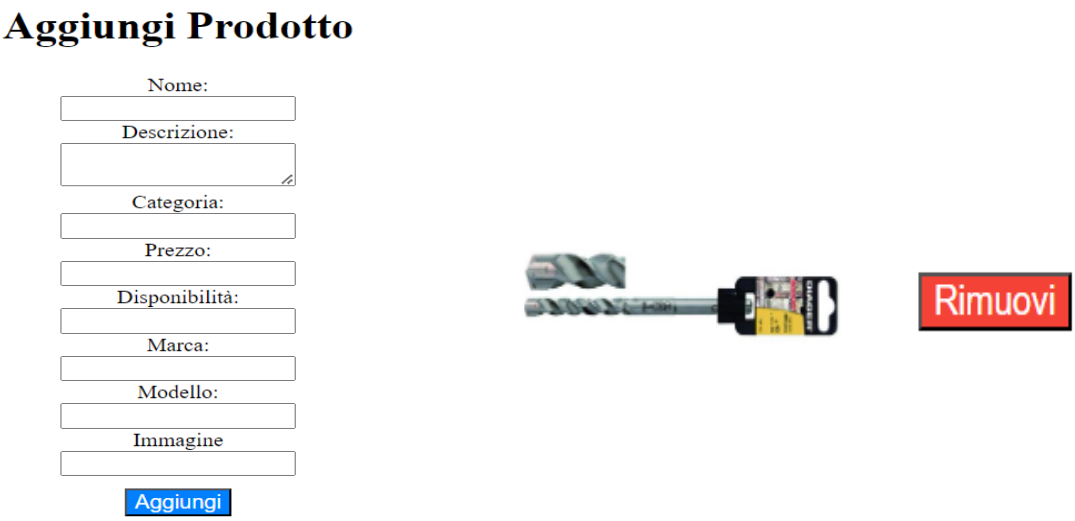
Nel progetto in questione sono presenti molte funzionalità:

- Possibilità di accesso tramite login se si è già registrati alla piattaforma altrimenti effettuare la registrazione inserendo i dati necessari e specificando se si è un venditore o meno.

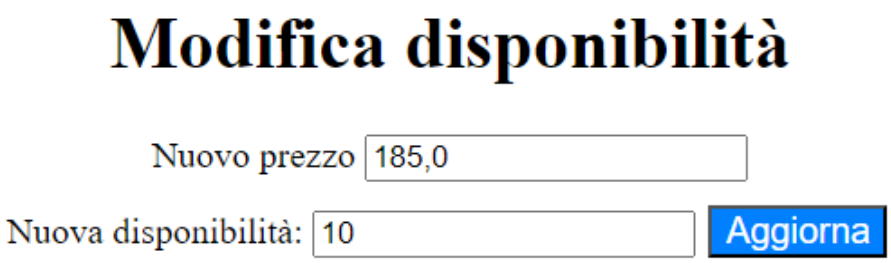


- **Possibilità per il venditore**:

  . Aggiungere prodotti inserendo marca, modello, quantità, prezzo e url dell'immagine e rimuoverli



  . Modificare le quantità e il prezzo degli elementi già presenti

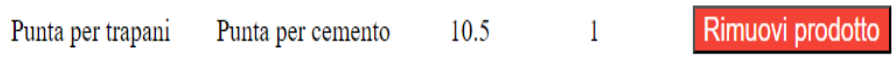


- **Possibilità per l'acquirente**:

  . Aggiungere prodotti al carrello



  . Visualizzare i prodotti del carrello e in caso rimuoverli



. Effettuare l'acquisto dei prodotti selezionati



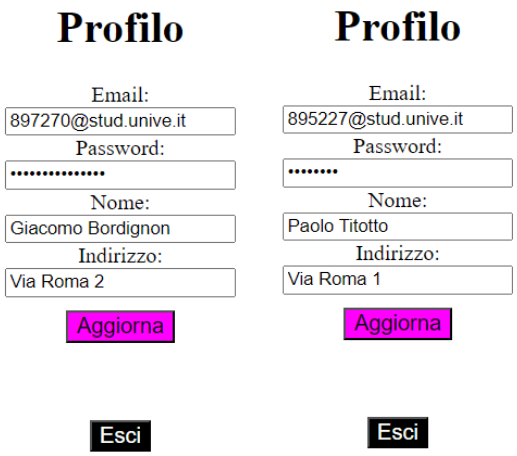
. Visualizzare lo storico degli ordini

Data	Stato	Totale	Nome	Modello
2024-07-23	Consegnato	450.0	trapano-avvitatore (Quantità: 1)	SB18LTBL
2024-07-23	Consegnato	217.0	trapano-avvitatore (Quantità: 1)	SB18L
2024-07-23	Consegnato	217.0	trapano-avvitatore (Quantità: 1)	SB18L
2024-07-24	Consegnato	450.0	trapano-avvitatore (Quantità: 1)	SB18LTBL
2024-07-29	Consegnato	10.5	Punta per trapani (Quantità: 1)	SDS-PLUS BOOSTER 6X160

. Effettuare ricerche attraverso parole chiavi per trovare più velocemente i prodotti desiderati

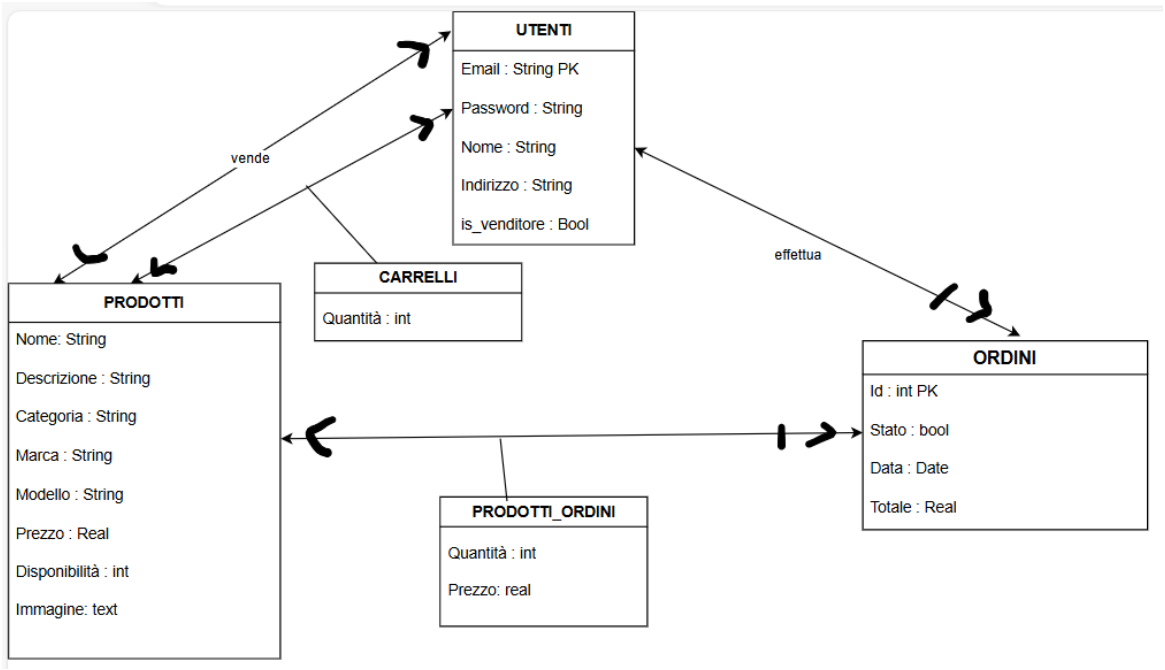


- Per entrambi c'è la possibilità di modificare il proprio profilo, partendo dalla email passando per la password, il nome e l'indirizzo. Sempre in questa pagina c'è anche la possibilità di effettuare il log-out



3. PROGETTAZIONE CONCETTUALE E LOGICA

Progettazione concettuale:



La progettazione concettuale è stata suddivisa in tre classi:

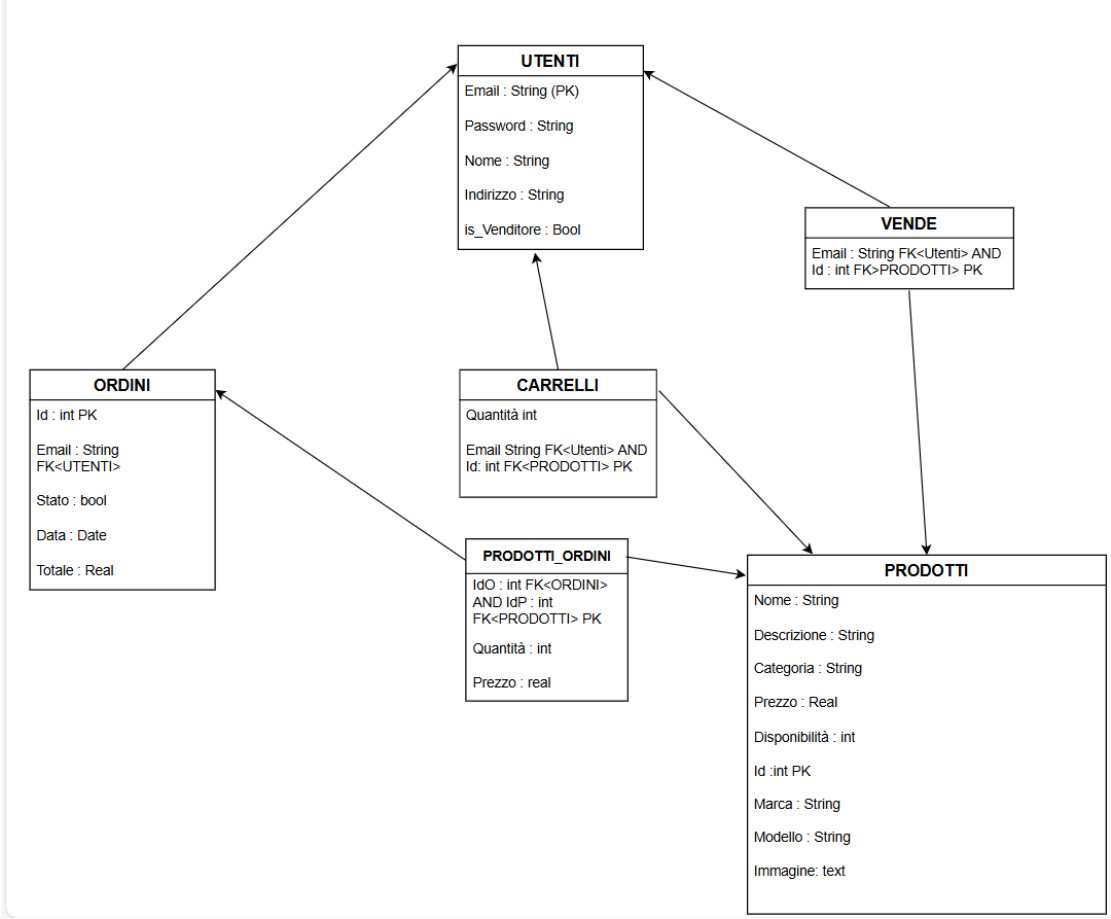
- **Utenti**: nella quale sono contenuti tutti i dati necessari tra cui anche se si tratta di un venditore o acquirente.
- **Prodotti**: sono presenti tutti i dati richiesti per trovare un prodotto, compresa l'immagine.
- **Ordini**: al suo interno troviamo la data in cui è stato effettuato l'ordine, il totale e soprattutto lo stato in cui si trova, quindi consegnato o meno.

I collegamenti sono strutturati come segue:

- **Vende**: collega Utenti a Prodotti con una molti-a-molti senza parzialità in quanto, secondo il nostro ragionamento, una persona può vendere più prodotti se pensiamo al venditore, oppure più prodotti possono essere venduti a più acquirenti.
- **Carrelli**: Collega Utenti a Prodotti. Si tratta di una relazione con attributo. Si tratta di una molti-a-molti per il ragionamento citato sopra, ma in aggiunta è presente l'attributo quantità che indica quanti prodotti sono stati messi nel carrello.
- **Effettua**: collega Utenti con Ordini. A differenza delle prime, questa è una relazione molti-a-uno con una parzialità dalla parte del molti. È stata strutturata nel seguente modo in quanto un ordine appartiene a un solo utente mentre un singolo utente può fare più ordini o anche nessuno in caso non ci sia la merce da lui cercata.

- **Prodotti_ordini:** collega Prodotti con ordini. Anche questa è una molti-a-molti ma con una parzialità su ordini e con degli attributi quali prezzo e la quantità di merce in fase di acquisto. Tutto questo perché gli ordini possono contenere più prodotti ma i prodotti possono essere contenuti in più ordini o anche in nessuno se un certo articolo non viene mai acquistato.

Progettazione logica:



La progettazione logica è il risultato della traduzione della progettazione concettuale utilizzando le tecniche apprese nel modulo 1 del corso.

Con questo sistema si sono create nuove classi:

- **Carrelli**: era l'attributo della relazione tra utenti e prodotti per quanto riguarda l'acquirente. Di conseguenza al suo interno, oltre alla quantità, sono presenti, sia come chiavi esterne che come primaria, gli attributi identificativi di utenti e prodotti.
- **Vende:** si è generata dalla relazione molti-a-molti e contiene l'email dell'utente e l'id del prodotto.
- **Prodotti_Ordini:** si è formata a partire dall'attributo della relazione tra ordini e prodotti, contiene le due chiavi esterne che sono anche primarie oltre alla quantità dei prodotti e il totale.

4. QUERY PRINCIPALI

Nel nostro progetto sono presenti diverse query importanti. Sono stati utilizzati attraverso codice python tramite SQLAlchemy e ORM.

SQLAlchemy:

Le troviamo in tutto il codice, alcune di queste sono risultate fondamentali per la ricerca, nel creare un nuovo ordine, nel visualizzare il carrello, nel visualizzare gli ordini.

- **Crea_ordine**: questa query ci permette ci aggiungere il nuovo ordine sia nella tabella Ordini sia in quella ProdottiOrdini in modo da poter vedere anche la cronologia degli acquisti effettuati

```
carrello = (db.session.query(Prodotti.id_Prodotti, Carrelli.quantita, Prodotti.prezzo).join(Prodotti,
                                                                                            Carrelli.id_Prodotti == Prodotti.id_Prodotti)
            .filter(Carrelli.email == session['email']).all())
totale = sum(item.quantita * item.prezzo for item in carrello)  # calcolo il totale dell'ordine
nuovo_ordine = Ordini(email=session['email'], stato=True, data=datetime.now(), totale=totale)  # creo l'ordine
db.session.add(nuovo_ordine)  # lo aggiungo alla tabella nel db
db.session.commit()  # eseguo il commit()

for item in carrello:
    prodOrdinati = ProdottiOrdini(id_Prodotti_=_item.id_Prodotti, id_Ordini_=_nuovo_ordine.id_Ordini,
                                  quantita_=_item.quantita, prezzo_=_item.prezzo)
    db.session.add(prodOrdinati)
db.session.commit()
```

- **Elimina_elementi_dal_carrello**: il codice in questione ci consente di eliminare i prodotti acquistati dal carrello in modo che non ci siano conflitti con l'inserimento di prodotti aggiuntivi prima di effettuare il pagamento

```
elimina_elementi = db.session.query(Carrelli).filter(session['email'] == Carrelli.email).all()
for item in elimina_elementi:  # elimino tutto ciò che è presente nel carrello con la mail di cui ho fatto il login
    db.session.delete(item)
db.session.commit()
```

- **Visualizza_carrello**: questa query è fondamentale per gli acquirenti che vogliono vedere tutto quello che stanno per acquistare, ovvero tutti prodotti presenti nel carrello, con la possibilità di rimuoverli

```
carrello = (
    db.session.query(Prodotti.id_Prodotti, Carrelli.quantita, Prodotti.nome, Prodotti.descrizione, Prodotti.prezzo)
    .join(Prodotti, Carrelli.id_Prodotti == Prodotti.id_Prodotti).filter(
        Carrelli.email == session['email']).all())
```

- **Ricerca**: aiuta l'acquirente a cercare più specificatamente determinati prodotti con la possibilità di inserire delle parole chiavi

```
prodotti = Prodotti.query.filter(
    Prodotti.nome.contains(parole_chiave) | Prodotti.descrizione.contains(parole_chiave)
    | Prodotti.modello.contains(parole_chiave)).all()
```

- **Visualizza_ordini**: ci permette di visualizzare lo storico degli ordini effettuati con relativi costi, quantità e nomi dei prodotti

```
ordini = db.session.query(Ordini, ProdottiOrdini, Prodotti).join(ProdottiOrdini,
        Ordini.id_Ordine == ProdottiOrdini.id_Ordini).join(Prodotti,
        ProdottiOrdini.id_Prodotti == Prodotti.id_Prodotti).filter(Ordini.email == session['email']).all()
#ordini = Ordini.query.filter_by(email=session['email']).all()
return render_template( template_name_or_list: 'ordini.html', ordini=ordini)
```

5. PRINCIPALI SCELTE PROGETTUALI

L'utilizzo dei ruoli (venditore e acquirente) sono stati gestiti tramite flask con una variabile di tipo bool is_venditore, se è True ci si riferisce al venditore altrimenti all'acquirente.

I vincoli di integrità sono stati garantiti tramite l'utilizzo di check e trigger nella parte SQL mentre nell'ambito python, tramite funzioni di SQLAlchemy unite a ORM, viene garantita la protezione da SQLInjection. Inoltre tramite l'utilizzo in html di {{ }}, dove vengono passati i dati del database, permettono la protezione da XSS.

CHECK:

- **Vincoli sugli utenti**: impediscono che la email e la password abbiamo una lunghezza inferiore a 6 caratteri

```
ALTER TABLE utenti
    ADD CONSTRAINT mail(length(email) >= 6)
    ADD CONSTRAINT password(length(password) >= 6)
```

- **Vincoli sui carrelli**: garantiscono che per procedere all'acquisto ci sia almeno un prodotto con quantità maggiore di 0

```
ALTER TABLE carrelli
    ADD CONSTRAINT qnt(quantita >= 0)
```

- **Vincoli sui ordini**: impedisce che ci sia un totale negativo da pagare

```
ALTER TABLE ordini
    ADD CONSTRAINT tot(totale > 0.00)
```

- **Vincoli sui prodotti**: permettono che non ci sia mai una quantità minore di 0 e lo stesso vale per il prezzo del prodotto in questione

```
ALTER TABLE prodotti
    ADD CONSTRAINT disp(disponibilita > 0)
    ADD CONSTRAINT prezzo(prezzo > 0)
```

TRIGGER:

Il trigger in questione serve per eliminare la righe della tabella carrelli che hanno lo stesso id_Prodotto

```
CREATE OR REPLACE FUNCTION eliminazione_dal_carrello()
RETURNS TRIGGER AS $$
BEGIN
    -- Elimina le righe dalla tabella carrelli che hanno lo stesso id_Prodotti
    DELETE FROM carrelli WHERE id_Prodotti = OLD.id_Prodotti;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER elimina_prodotti_carrello
BEFORE DELETE ON prodotti
FOR EACH ROW
EXECUTE FUNCTION eliminazione_dal_carrello()
```

INDICI:

Sono stati utilizzati gli indici nella tabella Prodotti, sono multi-valore su marca e modello con l'obbiettivo di velocizzare la ricerca

```
CREATE INDEX indici_prodotti
ON prodotti(marca,modello)
```

6. ULTERIORI INFORMAZIONI

Nel progetto E-commerce svolto sono state utilizzate molteplici librerie aggiuntive, grazie all'ausilio di alcuni pacchetti python, quali:

- Datetime
- psycopg2
- Flask
- Render_template
- Request
- Redirect
- url_for
- session
- SQLAlchemy

Sono stati utilizzati anche i linguaggi HTML e CSS per il front-end

7. CONTRIBUTO DEL PROGETTO

Il progetto è stato svolto, sia nella parte di back-end e front-end, da entrambi i membri del gruppo in ugual modo.